# Project Assignment 1

## Containerized Web Application with PostgreSQL using Docker Compose and Macvlan/Ipvlan

## Objective

Design, containerize, and deploy a web application using:

- **PostgreSQL (mandatory database)**
- Backend API using **either Node.js + Express OR FastAPI**
- Docker multi-stage builds
- Separate Dockerfiles (Backend + Database)
- Docker Compose for orchestration
- Macvlan or Ipvlan networking (mandatory)

> ***The system must demonstrate production-ready image building, container networking concepts, persistent storage, and service orchestration.***

## Project Deliverables

1. GitHub Repository
2. Separate Dockerfiles:
    - backend/Dockerfile
    - database/Dockerfile
3. docker-compose.yml
4. Network creation command
5. Screenshot proofs:
    - docker network inspect
    - container IPs
    - volume persistence test
6. Short report (3–5 pages):
    - Build optimization explanation
    - Network design diagram
    - Image size comparison
    - macvlan vs ipvlan comparison

2/7

# Mandatory Technology Stack

### Database (Compulsory)

- PostgreSQL

### Backend (Choose ONE)

- `Node.js + Express`
- `FastAPI`

### Networking

- Macvlan **OR**
- Ipvlan

3/7

# Architecture Requirements

Client (Browser or Postman) ↓ Backend Container (LAN IP via macvlan/ipvlan) ↓ PostgreSQL Container (static IP + named volume)

Constraints:

- Backend and Database must have separate Dockerfiles
- `docker-compose.yml` must orchestrate entire stack
- Database data must persist using named volume
- Static IP must be assigned using macvlan/ipvlan
- Containers must be reachable from LAN

4/7

# Functional Requirements

Backend must implement:

- POST endpoint → Insert record
- GET endpoint → Fetch records
- Healthcheck endpoint
- Database connection via environment variables
- Table auto-creation on startup

5/7

## Image Requirements (Strict)

Each Dockerfile must:

- Use multi-stage build wherever applicable
- Use minimal base image (alpine/slim)
- Include `.dockerignore`
- Use non-root user
- Avoid unnecessary layers
- Clearly separate builder and runtime stages

Database container:

- Must not use default postgres image directly without custom Dockerfile
- Must configure:
  - `POSTGRESDB`
  - `POSTGRES` `USER`
  - `POSTGRES_PASSWORD`

6/7

## Docker Compose Requirements

Compose file must:

- Define both services
- Use external macvlan or ipvlan network

- Assign static IP addresses
- Use named volume for Postgres
- Include restart policy
- Include healthcheck
- Use depends_on properly
- Pass environment variables securely

7/7

## Networking Requirements

- Create macvlan OR ipvlan network manually
- Configure subnet and gateway
- Assign static IPs
- Demonstrate LAN access
- Explain host isolation issue (macvlan case)